

Big Data Analytics - Assignment - 2

(III Year CSE - C & D) [Submission Due: 10/04/2026]

Unit-III:

Topics: Data formats, analyzing data with Hadoop, scaling out, Architecture of Hadoop distributed file system (HDFS), fault tolerance, with data replication, High availability, Data locality, Map Reduce Architecture, Process flow, Java interface, data flow, Hadoop I/O, data integrity, compression, serialization. Introduction to Hive, data types and file formats, HiveQL data definition, HiveQL data manipulation, Logical joins, Window functions, Optimization, Table partitioning, Bucketing, Indexing, Join strategies.

Important Questions:

- 1) Explain the Master-Slave architecture of HDFS with a detailed description of NameNode and DataNode roles.
- 2) What is the significance of the Secondary NameNode? Explain the process of merging FSImage and Edit Logs.
- 3) Discuss the HDFS Data Replication strategy and how Rack Awareness improves fault tolerance.
- 4) Elaborate on the MapReduce Architecture, detailing the functions of the JobTracker and TaskTracker.
- 5) Explain the step-by-step process flow of a MapReduce job, from Input Splits to the final Output.
- 6) Describe the concept of Data Locality and explain its three different levels (Node-local, Rack-local, and Off-switch).
- 7) What is the role of Serialization in Hadoop? Compare Java Serialization with Hadoop's Writable interface.
- 8) Discuss the importance of Data Integrity in HDFS and how Checksums are used to detect corruption.
- 9) Explain the different Compression codecs available in Hadoop and the impact of "splittable" formats on MapReduce.
- 10) Define the Architecture of Apache Hive and explain the role of the Metastore.
- 11) Compare and contrast Managed (Internal) Tables and External Tables in Hive.
- 12) Explain Hive Table Partitioning with an example. How does it optimize query performance?
- 13) What is Bucketing in Hive? Explain how it differs from Partitioning and when it should be used.
- 14) Discuss Hive Join Strategies, specifically focusing on the Map-side Join and Bucket Map Join optimizations.
- 15) Explain the HiveQL Data Manipulation Language (DML) operations, including LOAD DATA and INSERT OVERWRITE processes.

Unit-IV:

Topics: RDD, Transformations, DAG, SparkContext, SparkSession, DataFrames, Catalyst Optimizer, Joins, Window Functions, Grouping, Broadcast Variables, Accumulators, Deployment, Spark UI, Data Sources

Important Questions:

- 1) Explain the advantages of Apache Spark over Hadoop MapReduce. Illustrate the architectural differences with a suitable diagram.
- 2) What is an RDD (Resilient Distributed Dataset)? Explain its key properties and characteristics. Differentiate between Narrow and Wide Transformations with examples and diagrams.
- 3) Explain Spark's DAG (Directed Acyclic Graph) execution model. How does Spark convert transformations into Stages and Tasks? Illustrate with a diagram.
- 4) What are Spark DataFrames? Explain the Catalyst Optimizer and how it optimizes query execution. Include the optimization pipeline diagram.
- 5) Explain the different types of Joins supported in Spark DataFrames. Describe each join type with illustrative diagrams showing input and output.
- 6) Explain Grouping and Window Functions in Spark. How are Window Functions different from groupBy aggregations? Describe rank, dense_rank, lag, and lead functions.
- 7) Explain Broadcast Variables and Accumulators in Apache Spark. When are they used? What are their limitations?
- 8) Explain the different Spark deployment modes (Client and Cluster) and Cluster Managers — Standalone Mode and Spark on YARN — with architecture diagrams.
- 9) Explain the Spark Web UI in detail. What information does each tab provide? How does the Spark History Server work and why is it important?
- 10) Explain the different Data Sources supported in Apache Spark. How does Spark read from and write to various formats? Describe the role of lazy evaluation in data source reading.

Unit-V:

Topics: Performance Tuning, Stream Processing Fundamentals, Event-Time, Stateful Processing, Tumbling & Sliding Windows, Watermarks, Dropping Duplicates, Structured Streaming, Transformations, Input Sources, Output Sinks

Important Questions:

- 1) Explain Spark Performance Tuning. Discuss the key areas of tuning including partitioning, caching, serialization, and memory management.
- 2) What is Stream Processing? Explain the fundamental concepts of streaming systems. How is stream processing different from batch processing?
- 3) Explain Event-Time and Processing-Time in Spark Structured Streaming. Why is Event-Time preferred? Describe Tumbling Windows and Sliding Windows with diagrams.
- 4) What is Stateful Processing in Spark Structured Streaming? How does Spark maintain and manage state across micro-batches? Explain with use cases.
- 5) Explain Watermarks in Spark Structured Streaming. How do watermarks handle late-arriving data? Also explain how to drop duplicates in a streaming context.
- 6) Explain Spark Structured Streaming — its core concepts, the unbounded table model, and how it differs from Spark's legacy DStream API.
- 7) What transformations are supported in Spark Structured Streaming? Clearly explain which operations are NOT supported and why. Describe the output modes.
- 8) Explain all Input Sources supported in Spark Structured Streaming. Describe each source's characteristics, configuration options, and appropriate use cases.
- 9) Explain all Output Sinks supported in Spark Structured Streaming. Describe fault tolerance guarantees, output modes, and configuration of each sink.
- 10) Explain the end-to-end architecture of a Spark Structured Streaming application. Describe the role of each component — source, query, state store, checkpoint, sink — and explain fault tolerance mechanisms.

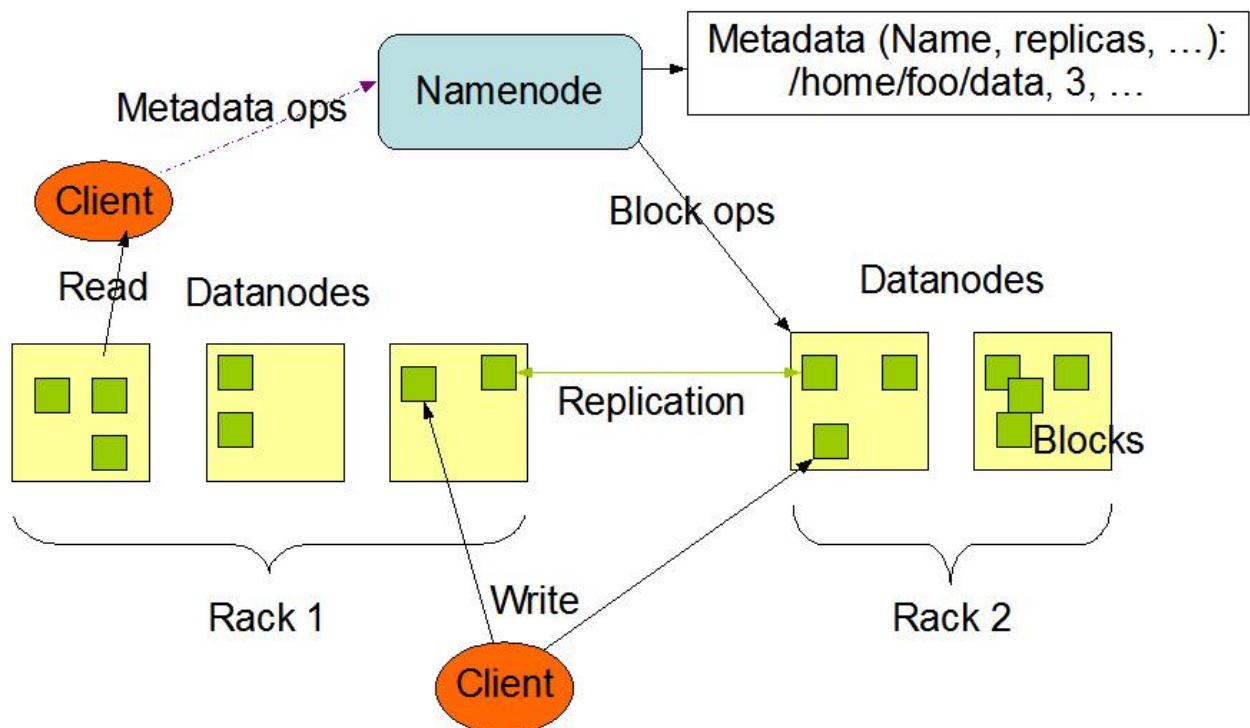
Unit-III: Answers (Only key points are provided in the answers, and students are expected to elaborate them thoroughly.)

Topics: Data formats, analyzing data with Hadoop, scaling out, Architecture of Hadoop distributed file system (HDFS), fault tolerance, with data replication, High availability, Data locality, Map Reduce Architecture, Process flow, Java interface, data flow, Hadoop I/O, data integrity, compression, serialization. Introduction to Hive, data types and file formats, HiveQL data definition, HiveQL data manipulation, Logical joins, Window functions, Optimization, Table partitioning, Bucketing, Indexing, Join strategies.

1) Explain the Master-Slave architecture of HDFS with a detailed description of NameNode and DataNode roles.

HDFS (Hadoop Distributed File System) follows a **Master-Slave architecture** designed to store very large datasets across multiple commodity machines.

HDFS Architecture



Architecture

Master → NameNode

Slaves → DataNodes

Role of NameNode

The **NameNode** is the master server that manages:

File system namespace
File names and directory structure

- Access permissions
- Mapping of file blocks to DataNodes
- Block replication decisions
- Metadata storage

Main Responsibilities

- Maintains **FSImage**
- Maintains **EditLog**
- Controls **block placement**
- Monitors DataNodes through **heartbeats**
- Receives **block reports**

Role of DataNode

The **DataNode** stores actual data blocks.

Responsibilities

- Stores file blocks physically
- Serves client read/write requests
- Sends heartbeats periodically
- Sends block reports
- Creates, deletes, replicates blocks

Advantages

- Centralized metadata management
- Distributed scalable storage
- Easy fault recovery
- High throughput

2) What is the significance of the Secondary NameNode? Explain the process of merging FSImage and Edit Logs

The **Secondary NameNode** is mainly used for **checkpointing**.
It is **NOT** a backup NameNode.

Why it is needed?

The NameNode continuously stores metadata updates in:

FSImage → snapshot of file system

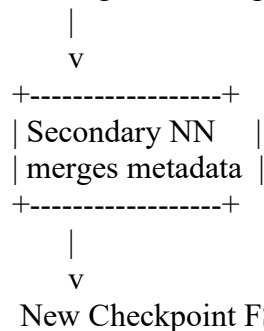
EditLog → recent metadata changes

Over time, EditLog becomes huge.

The Secondary NameNode periodically merges both.

Merging Process

FSImage + EditLog



Step-by-Step

Copies FSImage from NameNode

Copies EditLog

Loads both in memory

Merges updates

Creates new checkpoint

Sends back updated FSImage

Benefits

Reduces NameNode restart time

Prevents huge EditLog growth

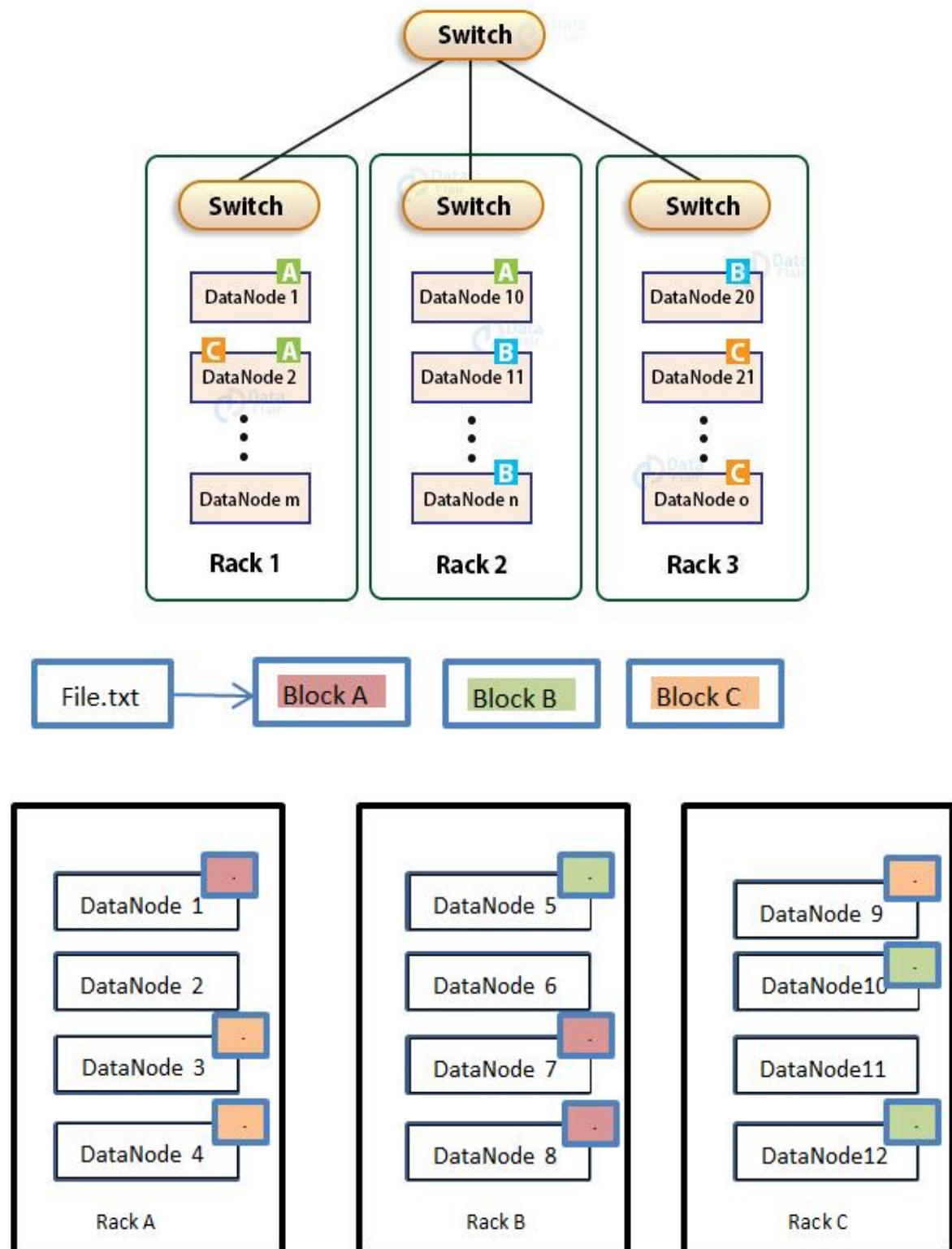
Improves metadata recovery

3) Discuss the HDFS Data Replication strategy and how Rack Awareness improves fault tolerance

HDFS stores multiple replicas of each block.

Default Replication Factor

3 replicas



Replication Strategy

For replication factor = 3:

Replica 1 → same node/local rack

Replica 2 → another node in same rack

Replica 3 → node in different rack

Rack 1: Node A, Node B

Rack 2: Node C

Block B1:

A → B → C

Rack Awareness

Rack awareness means Hadoop knows the network topology.

Why useful

If one rack fails:

another rack still contains data replica

Fault Tolerance Improvements

Protects against:

Node crash

Disk failure

Rack switch failure

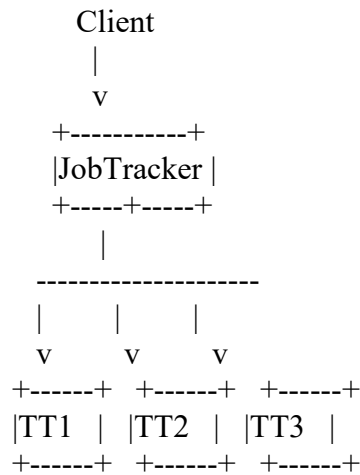
Complete rack power loss

This improves **availability and reliability**.

4) Elaborate on the MapReduce Architecture, detailing the functions of the JobTracker and TaskTracker

MapReduce uses **master–slave execution architecture**.

Architecture



JobTracker Functions

Receives job from client

Splits into tasks

Schedules map/reduce tasks

Tracks execution

Handles failed task re-execution

TaskTracker Functions

Executes map tasks

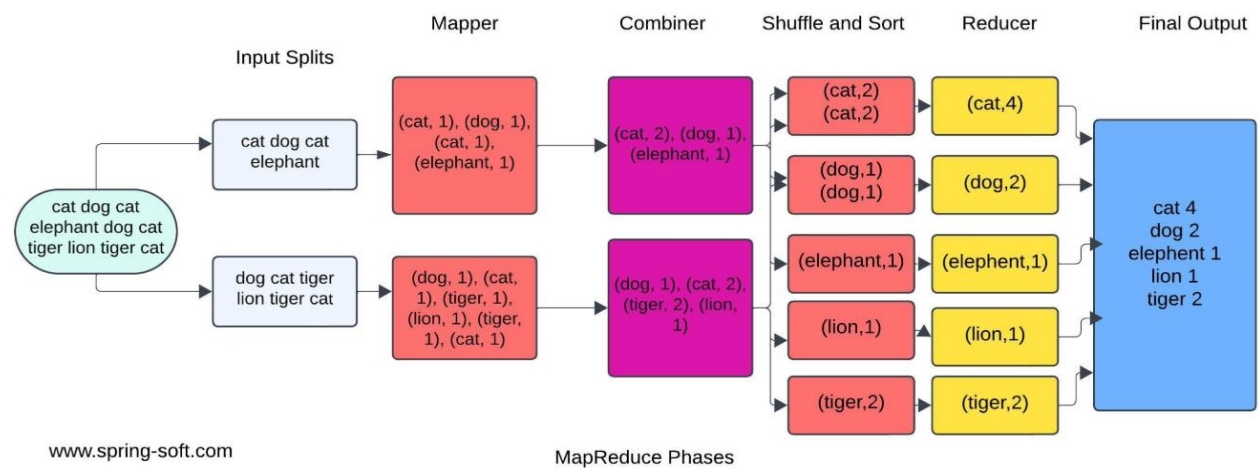
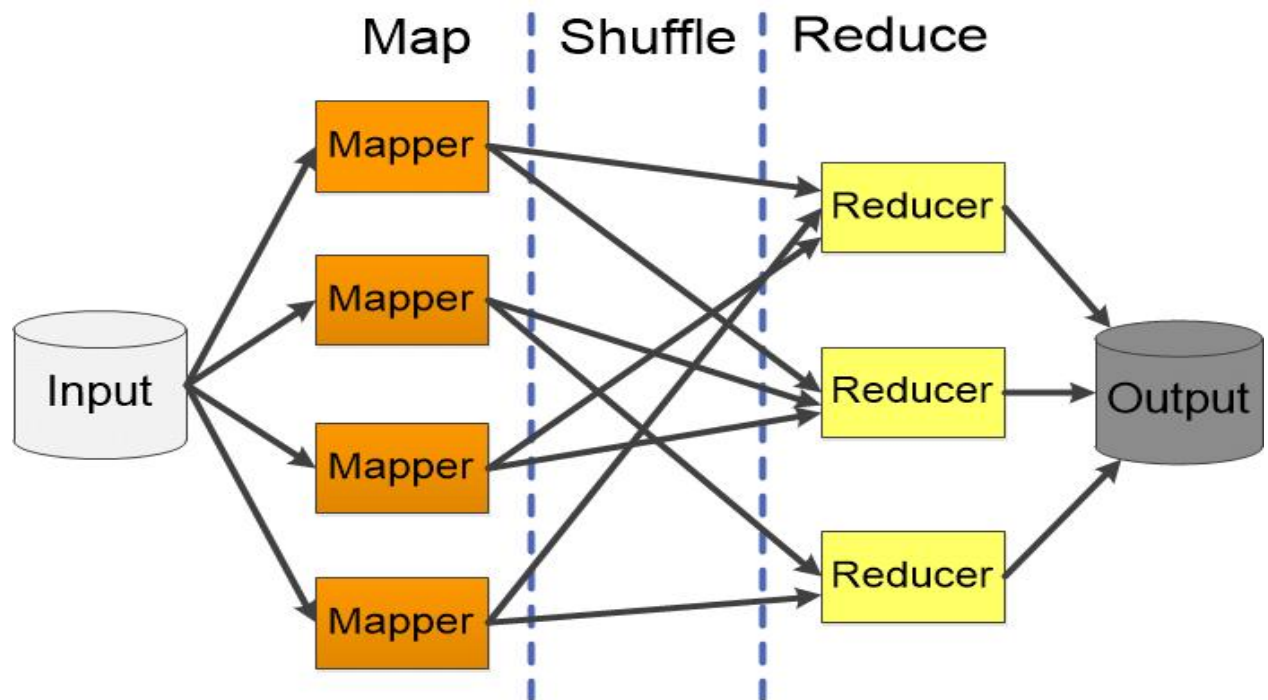
Executes reduce tasks

Sends status updates

Sends heartbeat

Stores intermediate output

5) Explain the step-by-step process flow of a MapReduce job



Process Flow

Input File

|
v

Input Split

|
v

Mapper

|
v

Shuffle & Sort

|
v

Reducer

|
v

Output File

Detailed Steps

Input file stored in HDFS

File divided into **Input Splits**

RecordReader converts into key-value pairs

Mapper processes each record

Intermediate key-value pairs created

Shuffle groups same keys

Sort arranges keys

Reducer aggregates values

Output written to HDFS

6) Describe Data Locality and its three levels

Data locality means **moving computation near data instead of moving data across network.**

Levels

1. Node-Local

Task runs on same machine as block.

Best performance

2. Rack-Local

Task runs on another node in same rack.

Moderate performance

3. Off-Switch

Task runs on node in different rack.

Higher latency

7) Role of Serialization in Hadoop

Serialization converts objects into byte streams.

Need

Data transfer over network

Disk storage

Mapper → Reducer communication

Java Serialization

Slower

More metadata overhead

Generic use

Writable Interface

Faster

Compact

Hadoop optimized

Examples

IntWritable

LongWritable

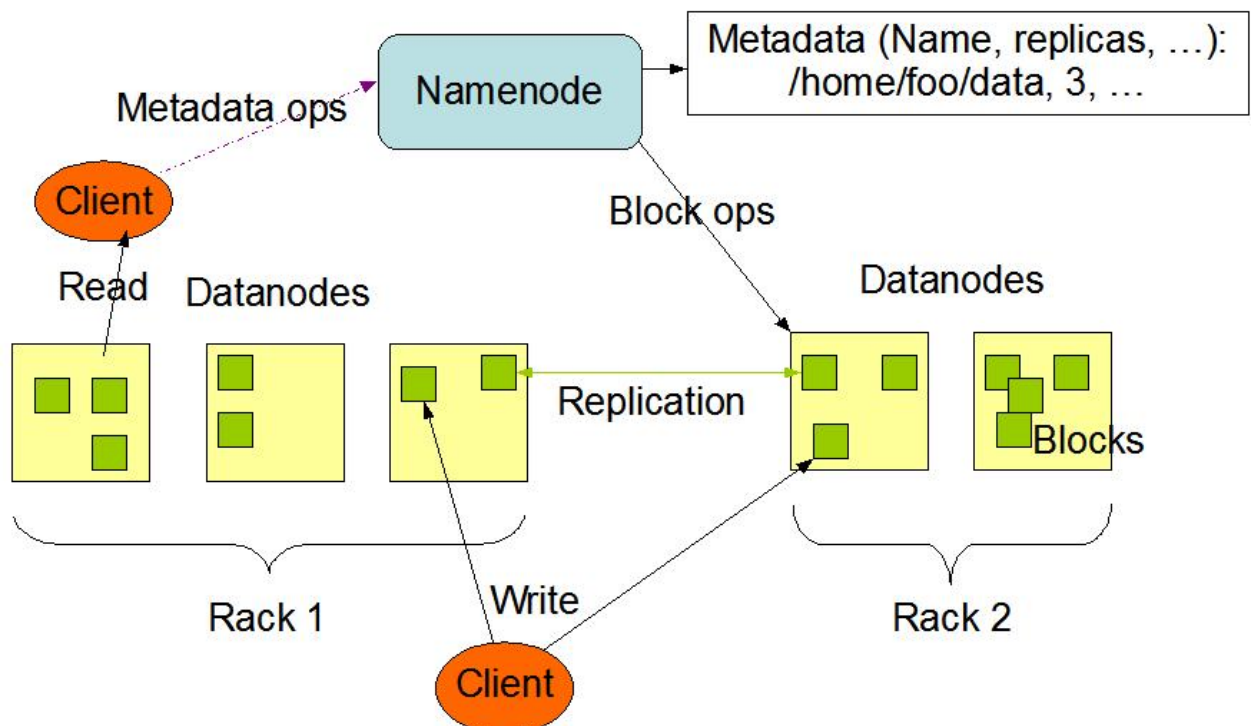
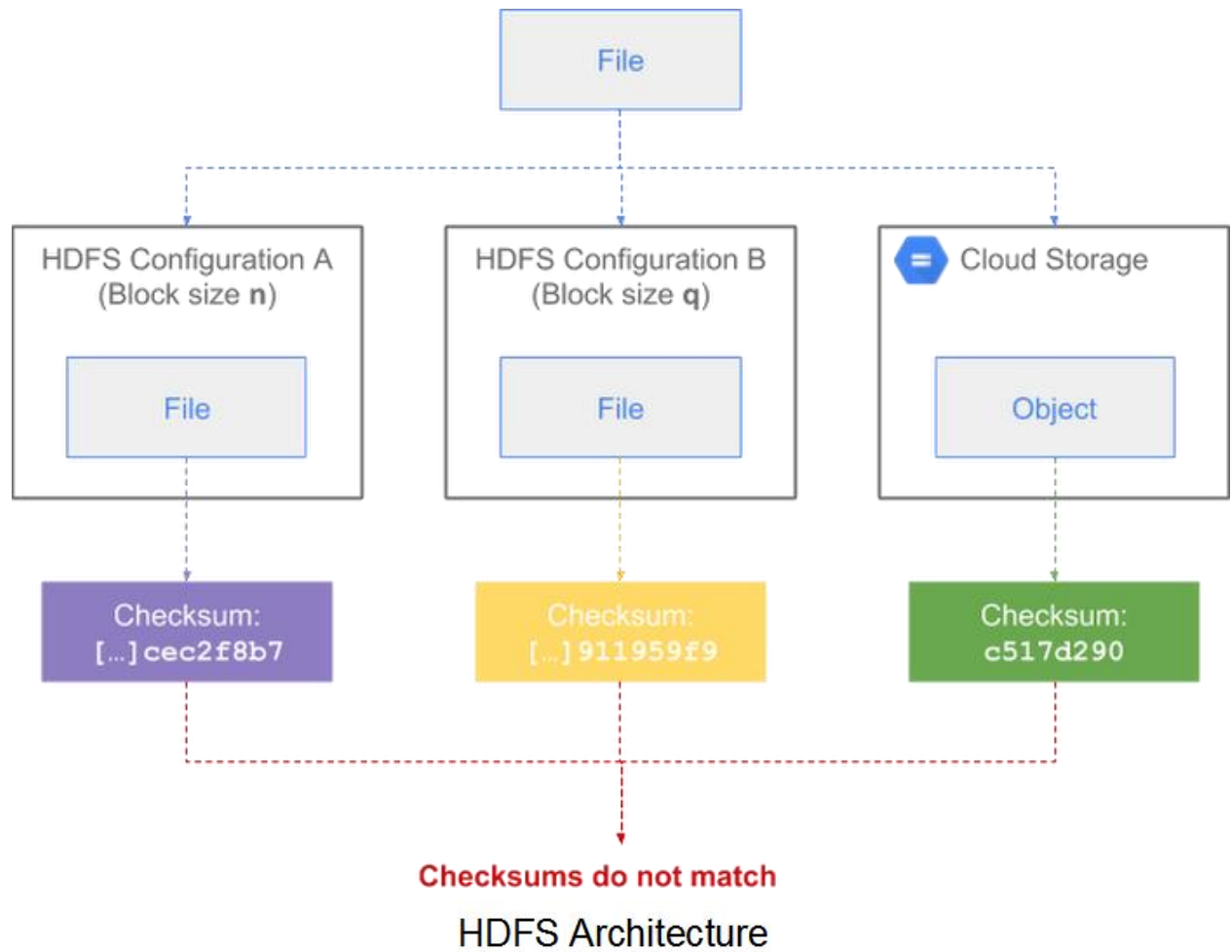
Text

DoubleWritable

Comparison

Feature	Java Serialization	Writable
Speed	Slow	Fast
Size	Large	Compact
Hadoop use	Not preferred	Preferred

HDFS ensures stored data remains uncorrupted.



Checksum Process

Checksum generated for each chunk

Stored separately

During read, checksum recalculated

Compared with stored checksum

If mismatch → corruption detected

Recovery

Client automatically reads another replica.

This ensures **high reliability**.

9) Hadoop Compression codecs and splittable formats

Codecs

Gzip → high compression, not splittable

Bzip2 → splittable

Snappy → fast

LZO → fast and splittable (with indexing)

Splittable Importance

If file is splittable:

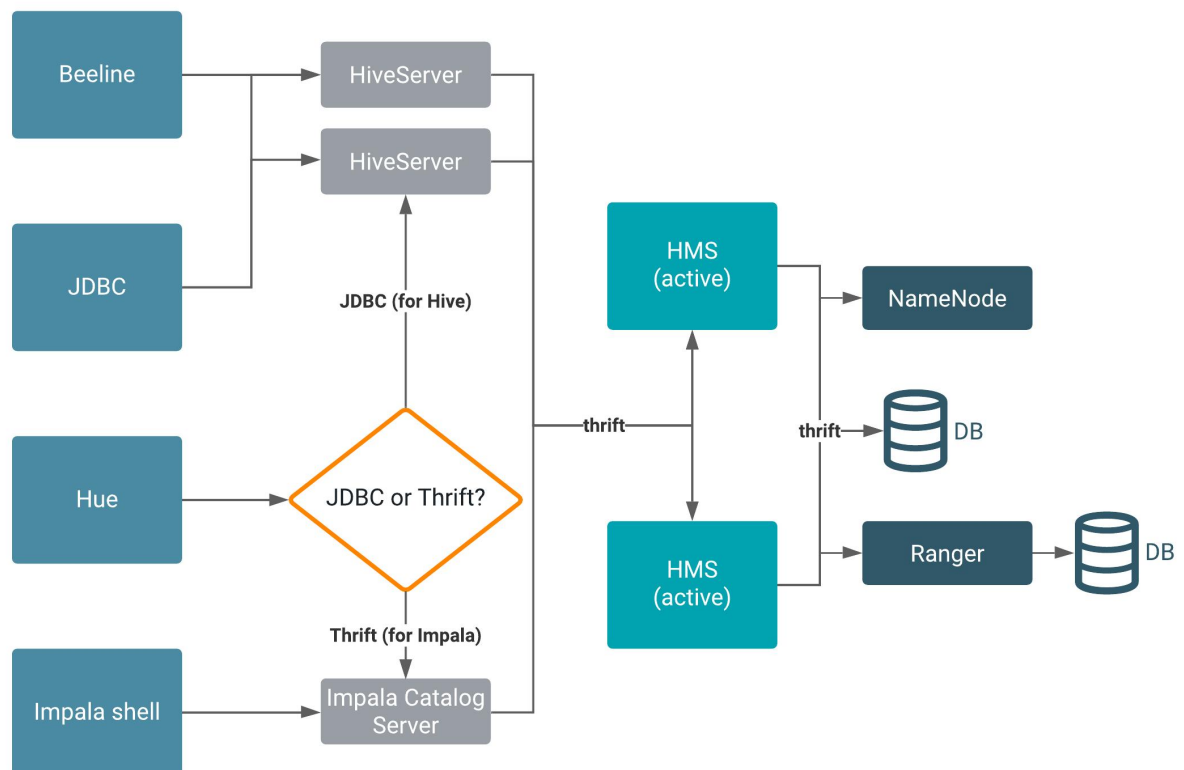
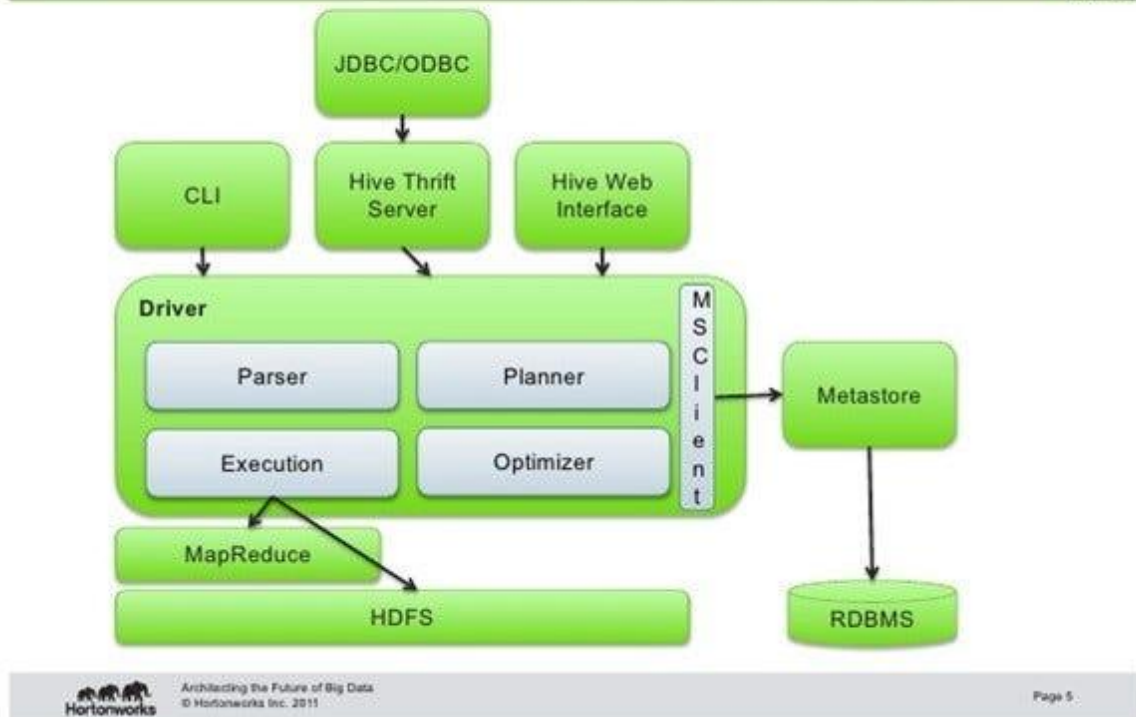
Multiple mappers can process simultaneously

Better parallelism

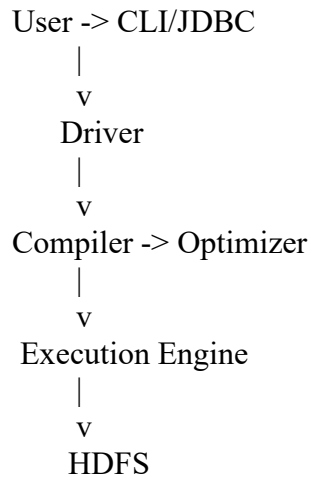
Faster MapReduce jobs

10) Apache Hive Architecture and Metastore

Apache Hive Architecture



Architecture



Metastore Role

Stores:

Table schema

Column data types

Partition info

Table location

SerDe info

Metastore is essential for **schema management**.

11) Managed vs External Tables in Hive

Managed Table

Hive manages data + metadata

DROP removes both

External Table

Hive manages only metadata

Actual data remains after DROP

Difference

Feature	Managed	External
Data ownership	Hive	User/HDFS
DROP behavior	deletes data	keeps data

12) Hive Partitioning with example

Partitioning divides data into subdirectories.

```
CREATE TABLE sales(  
  id INT,  
  amount DOUBLE  
)  
PARTITIONED BY (year INT, month INT);
```

Directory Structure

/year=2025/month=03/

Optimization

Query:

```
SELECT * FROM sales WHERE year=2025;
```

Only matching partition scanned.

Less I/O

Faster queries

13) What is Bucketing in Hive?

Bucketing divides data into fixed files using hash.
CLUSTERED BY (student_id)
INTO 8 BUCKETS;

Difference from Partitioning

Partitioning	Bucketing
directory based	file based
by column value	by hash
uneven possible	uniform

Use Cases

Sampling

Efficient joins

Better parallelism

14) Hive Join Strategies

Map-Side Join

Small table loaded into memory.

No reducer

Fast

Bucket Map Join

Both tables bucketed on join key.

Matching buckets directly joined

Less shuffle

Best for **large warehouse joins**.

15) HiveQL DML operations

LOAD DATA

```
LOAD DATA INPATH '/data/sales.csv'  
INTO TABLE sales;  
Loads HDFS/local data into Hive table.
```

INSERT OVERWRITE

```
INSERT OVERWRITE TABLE sales_summary  
SELECT region, SUM(amount)  
FROM sales  
GROUP BY region;
```

Process

Executes query

Replaces existing table data

Writes fresh output

Used in **ETL pipelines and batch refresh.**

Unit-IV: Answers (Only key points are provided in the answers, and students are expected to elaborate them thoroughly.)

Topics: RDD, Transformations, DAG, SparkContext, SparkSession, DataFrames, Catalyst Optimizer, Joins, Window Functions, Grouping, Broadcast Variables, Accumulators, Deployment, Spark UI, Data Sources

Q1. Explain the advantages of Apache Spark over Hadoop MapReduce. Illustrate the architectural differences with a suitable diagram.

Answer:

Introduction

Apache Spark is a fast, open-source, distributed computing framework designed for large-scale data processing. It overcomes the key limitations of Hadoop MapReduce by offering in-memory computation, a rich API, and a unified engine for multiple workloads.

Advantages of Apache Spark over Hadoop MapReduce

- 1. In-Memory Processing:** Spark stores intermediate results in RAM instead of writing them to HDFS after every Map and Reduce step. This makes Spark up to 100x faster than Hadoop for iterative workloads.
- 2. Lazy Evaluation:** Spark records all transformations as a Directed Acyclic Graph (DAG) and executes them only when an action is called. This enables the engine to optimize the execution plan before running.
- 3. Unified Platform:** Spark supports batch processing, SQL queries (Spark SQL), real-time streaming (Spark Streaming / Structured Streaming), machine learning (MLlib), and graph processing (GraphX) in a single framework. Hadoop requires separate tools for each.
- 4. Fault Tolerance via Lineage:** Spark recovers lost data using RDD lineage (a graph of transformations) without replicating data on disk, unlike Hadoop which relies on HDFS replication.
- 5. Rich APIs:** Spark offers APIs in Scala, Java, Python, and R. Hadoop MapReduce supports only Java natively.
- 6. Interactive Shell:** Spark provides an interactive REPL (spark-shell for Scala, pyspark for Python) for exploratory data analysis, which Hadoop lacks.
- 7. Better Real-Time Support:** Spark Streaming supports micro-batch real-time processing. Hadoop has no native streaming capability.

Architectural Comparison Diagram

The diagram below compares the two-stage disk-based MapReduce pipeline with Spark's in-memory DAG pipeline:

HDFS Input	Map Task	Shuffle/Sort	Reduce Task
Disk I/O at every step HDFS Write after Map HDFS Write after Reduce			

Figure 1a: Hadoop MapReduce – Disk-based two-stage pipeline

Data Source	RDD / DF	Transform	Transform	Action	Result
In-Memory at every step DAG Optimization Disk only when memory is full					

Figure 1b: Apache Spark – In-memory DAG-based pipeline

Comparison Table

Feature	Hadoop MapReduce	Apache Spark
Processing Model	Disk-based (two stages)	In-memory (multi-stage DAG)
Speed	Slower (100x slower than Spark)	Up to 100x faster for iterative jobs
Ease of Use	Java only, verbose code	Scala, Python, Java, R – concise
Real-time Support	No native support	Yes – Structured Streaming
Fault Tolerance	HDFS replication	RDD lineage graph
Use Cases	Simple batch processing	Batch, SQL, ML, Streaming, Graph

Q2. What is an RDD (Resilient Distributed Dataset)? Explain its key properties and characteristics. Differentiate between Narrow and Wide Transformations with examples and diagrams.

Answer:

Definition of RDD

An RDD (Resilient Distributed Dataset) is the fundamental data abstraction in Apache Spark. It represents an immutable, fault-tolerant, distributed collection of elements that can be processed in parallel across a cluster of machines.

Key Properties of RDD

- 1. Resilient (Fault-Tolerant):** If any partition of an RDD is lost due to a node failure, Spark can recompute it from its lineage (the sequence of transformations that created it) without re-reading from disk.
- 2. Distributed:** Data is automatically split into partitions and distributed across multiple nodes in the cluster, enabling parallel processing.
- 3. Immutable:** Once created, an RDD cannot be modified. Every transformation produces a new RDD. This immutability is key to fault tolerance.
- 4. Lazily Evaluated:** Transformations on an RDD are not executed immediately. Spark builds a logical execution plan (DAG) and executes it only when an action is called.
- 5. Typed:** RDDs are strongly typed (e.g., RDD[Int], RDD[String]) ensuring compile-time type safety in Scala/Java.
- 6. Partitioned:** Each RDD is divided into partitions. The number of partitions controls the degree of parallelism.

Narrow Transformations

In a Narrow Transformation, each partition of the child RDD depends on at most one partition of the parent RDD. No data movement (shuffle) across the network is required. These are fast and efficient.

Examples: map(), filter(), flatMap(), union(), mapPartitions()

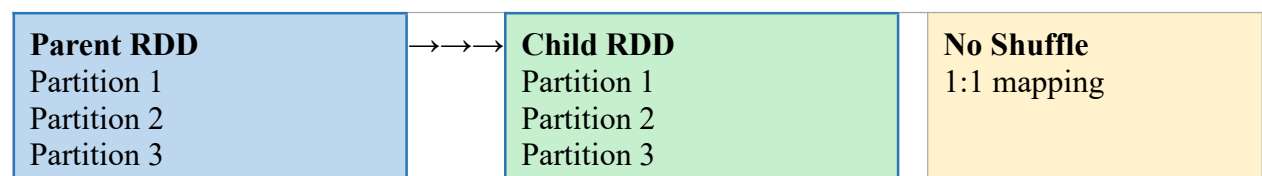


Figure 2a: Narrow Transformation – each child partition reads from exactly one parent partition

Wide Transformations

In a Wide Transformation, each partition of the child RDD may depend on multiple partitions of the parent RDD. Data must be shuffled (moved) across the network between nodes. These are expensive and mark stage boundaries.

Examples: groupByKey(), reduceByKey(), join(), distinct(), repartition(), sortByKey()

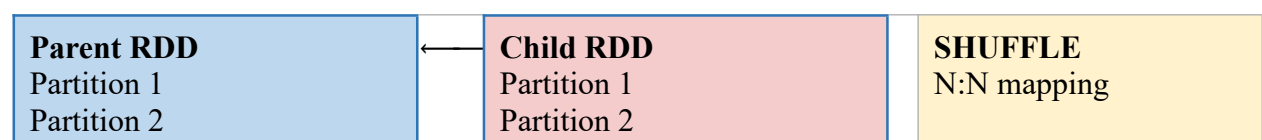


Figure 2b: Wide Transformation – child partitions depend on multiple parent partitions (data shuffle occurs)

Q3. Explain Spark's DAG (Directed Acyclic Graph) execution model. How does Spark convert transformations into Stages and Tasks? Illustrate with a diagram.

Answer:

What is a DAG in Spark?

A Directed Acyclic Graph (DAG) is a graph of vertices (RDD operations) and directed edges (dependencies between operations) with no cycles. In Spark, when a series of transformations is applied to data, the operations are not executed immediately. Instead, Spark builds a logical plan in the form of a DAG. This DAG is submitted to the DAG Scheduler for optimization and execution.

Why DAG instead of MapReduce?

Hadoop MapReduce forces all computations into two stages: Map and Reduce. The DAG model allows any number of stages with complex dependencies, enabling much more efficient execution plans — particularly for iterative algorithms.

DAG Execution Flow

The transformation pipeline is converted into an execution plan by the following components:

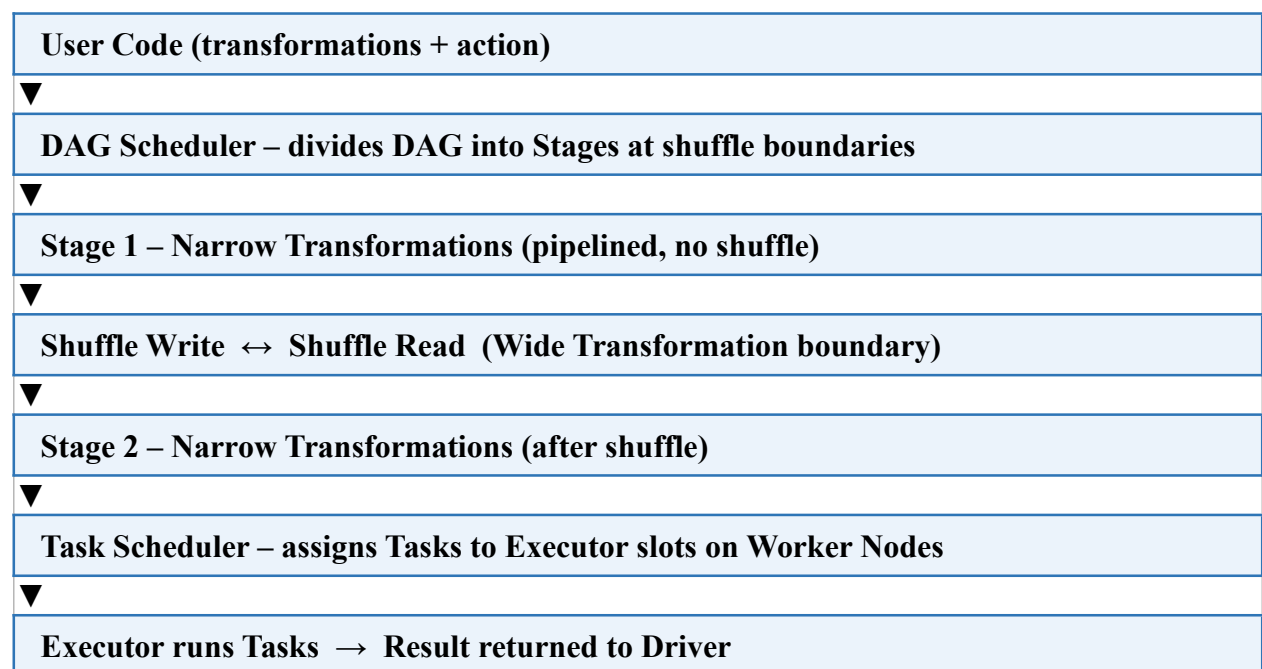


Figure 3: Spark DAG Execution Model – from User Code to Executor Tasks

Key Concepts

Job: Created when an action (count(), collect(), save()) is triggered. One job = one DAG.

Stage: A set of tasks with no shuffle boundary between them. Each wide transformation creates a new stage.

Task: The smallest unit of work. One task per partition per stage. Tasks run in parallel across executors.

DAG Scheduler: Converts the logical DAG into a physical execution plan (stages). It also optimizes for data locality.

Task Scheduler: Schedules individual tasks on executors based on availability and locality preferences.

Q4. What are Spark DataFrames? Explain the Catalyst Optimizer and how it optimizes query execution. Include the optimization pipeline diagram.

Answer:

Spark DataFrames

A DataFrame is a distributed collection of data organized into named columns — conceptually equivalent to a table in a relational database. Introduced in Spark 1.3, DataFrames provide a higher-level abstraction over RDDs with schema awareness and automatic optimization through the Catalyst Optimizer.

Advantages of DataFrames over RDDs

- Schema-aware: each column has a name and data type
- Automatically optimized by the Catalyst Optimizer
- Interoperable with SQL queries
- Efficient columnar storage via Project Tungsten
- Language-independent behavior (same performance in Python, Scala, Java, R)

The Catalyst Optimizer

The Catalyst Optimizer is the query optimization framework in Spark SQL. It applies rule-based and cost-based optimizations to transform a user query into the most efficient physical execution plan. It operates in four phases:

1. Parsing	Converts SQL string or DataFrame API calls into an Unresolved Logical Plan (AST)
2. Analysis	Resolves column names and table references using the Catalog. Produces a Resolved Logical Plan
3. Logical Optimization	Applies rule-based optimizations: predicate pushdown, constant folding, column pruning
4. Physical Planning	Generates one or more Physical Plans using execution strategies (e.g., BroadcastHashJoin vs SortMergeJoin)
5. Code Generation	Selects best Physical Plan and generates Java bytecode using Whole-Stage Code Generation

Figure 4: Catalyst Optimizer Pipeline – from Query to Optimized Physical Plan

Key Optimizations Applied by Catalyst

Predicate Pushdown: Filters are moved as close to the data source as possible, reducing the amount of data read.

Column Pruning: Only the columns required by the query are read from storage, minimizing I/O.

Constant Folding: Constant expressions are evaluated at compile time rather than repeatedly at runtime.

Join Reordering: Tables are reordered in joins to minimize intermediate result sizes.

Q5. Explain the different types of Joins supported in Spark DataFrames. Describe each join type with illustrative diagrams showing input and output.

Answer:

Spark DataFrames support all standard SQL join types. Joins combine two DataFrames based on a matching key column. The choice of join type determines which rows appear in the result.

Join Type	Description	Result Behavior
Inner Join	Returns only rows where the join key exists in BOTH DataFrames.	Rows without a match are excluded from both sides.
Left Outer Join	Returns all rows from the LEFT DataFrame and matched rows from the right. Unmatched right rows are NULL.	All left rows preserved; right side may have NULLs.
Right Outer Join	Returns all rows from the RIGHT DataFrame and matched rows from the left. Unmatched left rows are NULL.	All right rows preserved; left side may have NULLs.
Full Outer Join	Returns all rows from BOTH DataFrames. Unmatched rows from either side appear with NULLs.	All rows preserved; NULLs fill missing matches.
Left Semi Join	Returns only rows from the LEFT DataFrame that have a matching row in the right. Right columns are NOT included.	Filter on left using right as lookup; no right columns.
Left Anti Join	Returns rows from the LEFT DataFrame that do NOT have a match in the right DataFrame.	Opposite of semi join – unmatched left rows only.
Cross Join	Returns the Cartesian product – every row of the left paired with every row of the right.	$n \times m$ rows produced; no key condition needed.

Visual Illustration of Core Join Types

INNER JOIN	LEFT OUTER JOIN	FULL OUTER JOIN
Left $\cap$ Right Only matching rows	Left $\bigcirc \cap$ Right All left + match right	Left $\bigcirc \cup \bigcirc$ Right All rows both sides

Figure 5: Spark Join Types – conceptual Venn diagram representation

Spark Join Strategies (Physical Execution)

Broadcast Hash Join: When one DataFrame is small enough to fit in memory, it is broadcast to all executors. This avoids shuffle and is the fastest join strategy.

Sort-Merge Join: Default for large-large joins. Both DataFrames are sorted on the join key and then merged. Requires a full shuffle.

Shuffle Hash Join: One side is hashed into partitions and joined with the other. Faster than sort-merge for moderate sizes.

Q6. Explain Grouping and Window Functions in Spark. How are Window Functions different from groupBy aggregations? Describe rank, dense_rank, lag, and lead functions.

Answer:

Grouping in Spark DataFrames

The groupBy() operation groups rows that share the same value in one or more columns and then applies aggregate functions such as count, sum, avg, min, and max to each group. groupBy() reduces the number of rows — one output row per group.

Window Functions

Window Functions perform calculations across a set of rows related to the current row, without collapsing those rows into a single output. Unlike groupBy(), every input row produces an output row. The set of rows used for calculation is defined by a Window Specification.

Aspect	groupBy + Aggregation	Window Function
Output rows	One row per group (reduces rows)	One row per input row (preserves rows)
Access to individual rows	Lost after aggregation	Available alongside aggregate result
Use case	Totals, counts, averages per group	Ranking, running totals, lag/lead analysis
Example	Average salary per department	Rank each employee within department

Window Specification Components

PARTITION BY: Divides data into independent windows (like GROUP BY but without collapsing rows). Example: partition by department.

ORDER BY: Defines the ordering of rows within each partition. Required for ranking and lag/lead functions.

ROWS/RANGE BETWEEN: Defines the window frame — which rows relative to the current row to include in the calculation.

Common Window Functions

rank(): Assigns a rank to each row within a partition. Rows with equal values receive the same rank, and the next rank has a gap (1,2,2,4).

dense_rank(): Like rank() but without gaps in the ranking sequence (1,2,2,3).

row_number(): Assigns a unique sequential number to each row within a partition regardless of ties.

lag(column, n): Returns the value of a column from n rows before the current row. Useful for comparing current vs. previous period.

lead(column, n): Returns the value of a column from n rows after the current row. Useful for comparing current vs. future values.

Emp ID	Dept	Salary	rank()	dense_rank()	lag(Salary,1)
E1	HR	60000	1	1	NULL
E2	HR	55000	2	2	60000
E3	HR	55000	2	2	55000
E4	HR	50000	4	3	55000

Figure 6: Window Function output – rank, dense_rank, and lag applied within HR department partition

Q7. Explain Broadcast Variables and Accumulators in Apache Spark. When are they used? What are their limitations?

Answer:

Broadcast Variables

In Spark, every task receives a copy of the variables it uses. For large, read-only data (such as a lookup table or a configuration dictionary), this means sending the data to every executor for every task — which is highly inefficient. Broadcast Variables solve this problem by caching a read-only variable on each worker node once, rather than sending it with every task.

How Broadcast Variables Work



Figure 7a: Broadcast Variable – distributed once by Driver, cached on all Executors

Use Cases: Joining a large DataFrame with a small lookup table; distributing ML model parameters; sharing a country-to-timezone mapping across all tasks.

Limitation: Broadcast variables are read-only. They cannot be modified by tasks. The variable must fit in the memory of each executor.

Accumulators

Accumulators are write-only variables from the perspective of the executors. They are used to aggregate information across tasks (such as counters or sums) and make the accumulated value available to the driver. They implement a distributed counter pattern.

How it works: Tasks can only add to an accumulator (using associative and commutative operations). Only the driver can read the final accumulated value.

Types: Numeric (LongAccumulator, DoubleAccumulator) and custom accumulators using AccumulatorV2.

Use Cases: Counting malformed records in a dataset; counting events meeting a condition; tracking custom metrics without a separate aggregation job.

Critical Limitation: Accumulators inside transformations may be updated more than once if tasks are re-executed (due to retries or speculative execution). Accumulators are reliable and counted exactly once only inside actions (foreach, foreachPartition).

Q8. Explain the different Spark deployment modes (Client and Cluster) and Cluster Managers — Standalone Mode and Spark on YARN — with architecture diagrams.

Answer:

Spark Deployment Modes

Client Mode: The Spark driver runs on the machine that submits the application (the client machine). The driver communicates with executors on the cluster. This mode is suitable for interactive and debugging use cases but requires the client machine to remain connected for the entire job duration.

Cluster Mode: The driver runs on one of the worker/cluster nodes, managed by the cluster manager. This is the preferred mode for production workloads since the client machine can disconnect after submission.

Aspect	Client Mode	Cluster Mode
Driver location	On the submitting machine	On a cluster worker node
Best for	Interactive development, debugging	Production batch jobs
Network dependency	Client must stay connected	Client can disconnect after submit
Log access	Directly on client terminal	Via YARN/cluster UI or log files

Cluster Manager 1: Standalone Mode

Standalone Mode is Spark's own built-in, lightweight cluster manager. It does not require Hadoop or any external resource manager. It is the simplest way to set up a Spark cluster.

Master Node (Resource Manager + Cluster UI on port 8080)		
Registers and monitors Workers		
Worker Node 1	Worker Node 2	Worker Node 3
<i>Each Worker Node launches Executors (JVM processes with CPU + Memory)</i>		

Component	Role in Standalone Mode
Master Node	Accepts job submissions, schedules executors, monitors workers
Worker Node	Registers with Master, launches Executors for Spark applications
Executor	JVM process on worker – runs tasks and stores data for the application
Driver	Submits the application to the Master; monitors task execution via DAG

Cluster Manager 2: Spark on YARN

YARN (Yet Another Resource Negotiator) is Hadoop's resource management layer. Running Spark on YARN allows it to share cluster resources with other Hadoop workloads (HDFS, Hive, HBase, etc.) and enables centralized resource management.

YARN Component	Spark Counterpart	Role
ResourceManager	Cluster Master	Manages and allocates cluster-wide resources
NodeManager	Worker Node	Manages resources on each node; launches containers
ApplicationMaster	Spark Driver (in cluster mode)	Negotiates resources from RM; manages executors
Container	Executor	A unit of resource (CPU + Memory) running tasks

Q9. Explain the Spark Web UI in detail. What information does each tab provide? How does the Spark History Server work and why is it important?

Answer:

The Spark Web UI

The Spark Web UI is a browser-based interface for monitoring and debugging Spark applications in real time. By default, it is accessible on port 4040 of the machine where the driver is running. When multiple applications run simultaneously, they use ports 4041, 4042, and so on.

Tab	Information Provided	Key Use
Jobs	All jobs submitted by the application; status (running/succeeded/failed); number of stages and tasks; duration and start time	Identify slow or failed jobs
Stages	Details of each stage: number of tasks, input/output size, shuffle read/write bytes, task duration distribution (min/median/max), skew indicators	Diagnose data skew and bottlenecks
Storage	Cached RDDs and DataFrames: storage level (MEMORY_ONLY, MEMORY_AND_DISK), fraction cached, partition count, size in memory and on disk	Check if caching is effective
Environment	Spark configuration properties, Java/Scala version, classpath entries, system properties	Verify configuration settings
Executors	All executors: ID, address, state, memory used/total, disk used, active tasks, completed tasks, failed tasks, GC time	Identify overloaded executors or GC issues
SQL / DataFrame	Executed SQL queries and DataFrame operations; logical and physical query plans; execution metrics per node	Understand query plan and optimize joins

Spark History Server

Once a Spark application finishes, its Web UI (on port 4040) is no longer accessible. The Spark History Server provides a persistent, post-execution interface to review application logs for completed (and in some cases, still-running) applications.

How it works: The application writes event logs to a configured directory (HDFS, S3, or local filesystem) during execution. The History Server reads these event logs and reconstructs the UI, making all tabs (Jobs, Stages, Storage, Executors, SQL) available for post-mortem analysis.

Default port: The History Server runs on port 18080.

Configuration required: `spark.eventLog.enabled = true` and `spark.eventLog.dir = <path>` must be set before the application runs.

Importance: Essential for debugging production jobs that run overnight or in cluster mode where the driver logs are not directly visible. Enables performance comparison across multiple application runs.

Q10. Explain the different Data Sources supported in Apache Spark. How does Spark read from and write to various formats? Describe the role of lazy evaluation in data source reading.

Answer:

Spark Data Sources Overview

Spark provides a unified Data Source API (through `spark.read` and `DataFrame.write`) to interact with a wide variety of storage formats and systems. This API abstracts the underlying storage details, allowing developers to work with a consistent interface regardless of the data source.

Data Source	Format Type	Key Characteristics
CSV	Text (row-based)	Human-readable, widely used, supports schema inference, header option, delimiter customization
JSON	Text (row-based)	Schema inferred automatically, supports nested/complex types, multiline JSON option
Parquet	Binary (columnar)	Default Spark format; highly compressed; supports predicate pushdown and column pruning; schema evolution
ORC	Binary (columnar)	Optimized for Hive; excellent compression; supports ACID transactions in Hive
Avro	Binary (row-based)	Schema stored with data; excellent for Kafka integration; supports schema evolution
JDBC	Relational DB	Read/write from MySQL, PostgreSQL, Oracle, SQL Server via JDBC drivers; supports partitioned reads
Hive Tables	Managed/External	Accessed via <code>spark.sql()</code> ; integrates with Hive metastore; supports partitioning and bucketing
Delta Lake	Binary (columnar)	ACID transactions, time travel, schema enforcement, built on Parquet

Read and Write Modes

Read API pattern: `spark.read.format('format').option('key','value').load('path')` – universally applicable to all formats

Write API pattern: `df.write.format('format').mode('mode').option('key','value').save('path')`

Write Mode	Behavior
overwrite	Deletes existing data and writes new data
append	Adds new data to existing data without deleting anything
ignore	Does nothing if data already exists (no error)
error / errorifexists	Raises an error if data already exists (default behavior)

Role of Lazy Evaluation in Data Source Reading

When `spark.read` is called, Spark does NOT immediately read any data. It only creates a logical plan describing how to read the data. The actual physical reading happens only when an action (`count()`, `show()`, `collect()`, `write()`) is triggered. This lazy evaluation allows Spark to:

- Apply predicate pushdown — filter conditions are pushed to the storage layer before reading
- Perform column pruning — only required columns are read from columnar formats like Parquet
- Optimize join ordering and scan operations before execution begins

Unit-V: Answers (Only key points are provided in the answers, and students are expected to elaborate them thoroughly.)

Topics: Performance Tuning, Stream Processing Fundamentals, Event-Time, Stateful Processing, Tumbling & Sliding Windows, Watermarks, Dropping Duplicates, Structured Streaming, Transformations, Input Sources, Output Sinks

Q1. Explain Spark Performance Tuning. Discuss the key areas of tuning including partitioning, caching, serialization, and memory management.

Answer:

Introduction to Performance Tuning

Performance tuning in Apache Spark involves identifying and eliminating bottlenecks in resource usage, data processing, and execution strategy to reduce job completion time and operational cost. Tuning operates at multiple levels: hardware resources, Spark configuration, data organization, and application design.

1. Partitioning

Partitions are the basic unit of parallelism in Spark. Each partition is processed by one task on one executor core. Poorly chosen partition counts lead to underutilization or excessive overhead.

Too few partitions: Cluster cores are idle; large tasks take longer; risk of OOM errors per task.

Too many partitions: Task scheduling overhead; many small shuffles; coordinator bottleneck.

Recommended formula: 2 to 4 partitions per CPU core in the cluster.

repartition(n): Reshuffles all data into exactly n partitions. Full shuffle occurs. Used to increase or decrease partition count.

coalesce(n): Reduces partitions without a full shuffle by merging local partitions. More efficient than repartition for reduction only.

spark.sql.shuffle.partitions: Default is 200. Should be reduced for small datasets (e.g., 20–50) and increased for very large ones.

2. Caching and Persistence

When an RDD or DataFrame is used multiple times, re-computing it from source is wasteful. Caching stores the computed result in memory (and optionally disk) so subsequent uses are fast.

Storage Level	Description
MEMORY_ONLY	Store in memory as deserialized Java objects. Fastest access. Recomputed if memory is insufficient.
MEMORY_AND_DISK	Store in memory; spill to disk if memory is full. More reliable than MEMORY_ONLY for large datasets.
DISK_ONLY	Store only on disk. Slowest access but does not consume RAM.
MEMORY_ONLY_SER	Store as serialized bytes in memory. Uses less RAM but requires deserialization time.
OFF_HEAP	Store in off-heap memory (Project Tungsten). Avoids Java GC overhead.

3. Serialization

Java Serialization: Default. Uses Java ObjectOutputStream. Flexible but slow and produces large serialized objects.

Kryo Serialization: Significantly faster and more compact than Java serialization. Recommended for performance-critical applications. Set via spark.serializer property.

4. Memory Management

Unified Memory Model (Spark 1.6+): Execution memory (shuffles, sorts, joins) and Storage memory (caching) share a common memory pool and can borrow from each other.

spark.memory.fraction: Fraction of total heap allocated to Spark (default 0.6). The remaining 0.4 is reserved for user data structures and metadata.

GC Tuning: Excessive Java Garbage Collection is a common bottleneck. Using serialized caching (MEMORY_ONLY_SER) or off-heap storage reduces GC pressure.

Q2. What is Stream Processing? Explain the fundamental concepts of streaming systems. How is stream processing different from batch processing?

Answer:

What is Stream Processing?

Stream processing is the continuous, real-time processing of data as it arrives, event by event or in small micro-batches. Unlike batch processing, which first stores data and then processes it periodically, stream processing produces results with very low latency — often within milliseconds to seconds of data arrival.

Batch vs Stream Processing

Aspect	Batch Processing	Stream Processing
Data collection	Accumulated over a period (hours/days)	Continuous arrival (unbounded stream)
Processing trigger	Scheduled (e.g., nightly job)	Continuous or near-continuous
Latency	High (minutes to hours)	Low (milliseconds to seconds)
Output freshness	Stale (reflects past state)	Fresh (near real-time)
Data volume	Bounded (finite dataset)	Unbounded (infinite stream)
Fault recovery	Re-run the batch job	Checkpointing and replay from source
Typical use case	Monthly reports, ETL pipelines	Fraud detection, live dashboards, IoT
Spark tool	Spark batch (RDD/DataFrame)	Spark Structured Streaming / DStream

Key Concepts in Streaming Systems

Source: The origin of streaming data. Examples: Apache Kafka, Amazon Kinesis, TCP socket, file directory, IoT sensors.

Sink / Output: Where processed streaming results are written. Examples: Kafka topics, HDFS/S3 files, databases, dashboards.

Trigger: Defines when Spark processes each micro-batch. Can be time-based (every 5 seconds), once (process available data and stop), or continuous (millisecond latency).

Checkpoint: Saves the current state and source offsets to fault-tolerant storage (HDFS/S3) after each micro-batch. Enables exactly-once processing and recovery after failure.

Watermark: A threshold that tells Spark how late data can arrive and still be included in a window. Data arriving after the watermark threshold is dropped.

State: Information maintained across micro-batches. Required for aggregations, sessionization, and deduplication.

Stream Processing Models in Spark

Micro-Batch Model: Spark collects incoming data into small batches at fixed time intervals and processes each batch using the Spark batch engine. Low latency (seconds). Default model in Structured Streaming.

Continuous Processing Model: Experimental mode with sub-millisecond latency. Records are processed immediately as they arrive, without batching.

Q3. Explain Event-Time and Processing-Time in Spark Structured Streaming. Why is Event-Time preferred? Describe Tumbling Windows and Sliding Windows with diagrams.

Answer:

Processing Time

Processing time is the time at which the Spark engine actually processes a record — i.e., the system clock of the Spark executor at the moment of computation. It is simple to implement but unreliable for out-of-order or late data since a late-arriving event would be assigned to the wrong time window.

Event Time

Event time is the timestamp embedded in the data itself, recording when the event actually occurred at the source (e.g., a sensor reading at 10:00 AM, a user click at 10:02 AM). Even if this data arrives late (at 10:15 AM), Spark can correctly assign it to the 10:00 AM window based on its event time.

Aspect	Processing Time	Event Time
Definition	When Spark processes the record	When the event actually occurred
Clock	Spark executor system clock	Timestamp embedded in the data
Out-of-order handling	Cannot handle; assigns wrong window	Correct window assignment regardless of arrival
Late data handling	Not handled	Handled via Watermarks
Use case	Approximate metrics, monitoring	Accurate analytics, billing, SLAs

Tumbling Windows

Tumbling windows are fixed-size, non-overlapping time windows. Each event belongs to exactly one window. The window size equals the slide interval.

10:00–10:05	10:05–10:10	10:10–10:15	10:15–10:20	10:20–10:25	10:25–10:30
<i>Each window is independent – no overlap. Window size = 5 minutes. Events belong to exactly ONE window.</i>					

Figure: Tumbling Windows (5-minute) – non-overlapping, contiguous time intervals

Sliding Windows

Sliding windows are overlapping windows of a fixed size that advance by a defined slide interval (which is smaller than the window size). An event may belong to multiple windows simultaneously.

10:00	10:02	10:04	10:06	10:08	10:10	10:12	10:14	10:16
W1 ←———— — 10 min — ————→				W2 ←———— — 10 min — ————→				
Window size = 10 min, Slide = 4 min → W1 [10:00–10:10], W2 [10:04–10:14], W3 [10:08–10:18] ... Events from 10:04–10:10 belong to BOTH W1 and W2.								

Figure: Sliding Windows – overlapping windows; events in the overlap zone belong to multiple windows

Property	Tumbling Window	Sliding Window
Overlap	No overlap	Overlap between consecutive windows
Event membership	Each event in exactly one window	Event may appear in multiple windows
Window count	Fewer windows	More windows (higher computation)
Use case	Non-overlapping summaries time	Rolling averages, trend detection

Q4. What is Stateful Processing in Spark Structured Streaming? How does Spark maintain and manage state across micro-batches? Explain with use cases.

Answer:

Stateless vs Stateful Processing

Stateless Processing: Each micro-batch is processed independently. No information is retained between batches. Examples: filtering rows, selecting columns, simple transformations. Fast and simple.

Stateful Processing: State (accumulated information) is maintained across multiple micro-batches. The result of processing each micro-batch depends on previously seen data. Required for aggregations, sessionization, deduplication, and pattern detection over time.

How Spark Maintains State

Spark uses an internal State Store (an in-memory key-value store, backed by checkpoints on HDFS/S3) to persist state between micro-batches. For each unique group key, the state store holds the aggregated value computed so far.

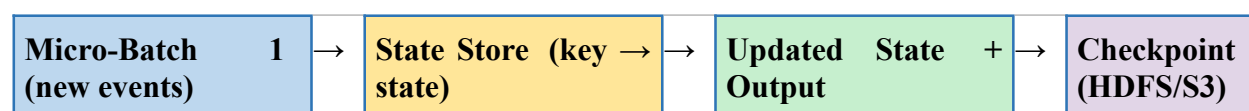


Figure: Stateful Processing – State Store persists between micro-batches and is checkpointed

Types of Stateful Operations

Windowed Aggregations: Aggregates computed within time windows (tumbling/sliding). State is bounded by the window size + watermark.

Unbounded Aggregations: Running totals or counts since stream start. State grows indefinitely. Requires careful memory management.

mapGroupsWithState: Allows arbitrary, user-defined stateful logic per group with full control over state updates, timeouts, and output.

flatMapGroupsWithState: Like mapGroupsWithState but can emit zero or more output rows per group per micro-batch.

Use Cases of Stateful Processing

- Session analysis – tracking a user's activity from login until inactivity timeout
 - Running totals – cumulative purchase amount per customer across time
 - Fraud detection – identifying suspicious transaction patterns over a sliding window
 - IoT monitoring – triggering alerts when sensor readings exceed a threshold for N consecutive intervals
-

Q5. Explain Watermarks in Spark Structured Streaming. How do watermarks handle late-arriving data? Also explain how to drop duplicates in a streaming context.

Answer:

The Problem of Late Data

In real-world streaming systems, events do not always arrive in the order they were generated. Network delays, device buffering, and clock skew cause events to arrive late. Without a mechanism to handle late data, either: (a) results are incorrect (late events ignored), or (b) state is held indefinitely (memory exhaustion).

What is a Watermark?

A watermark is a dynamic threshold that defines the maximum delay Spark is willing to tolerate for late data. It tells the engine: 'Do not wait for data that is older than (maximum observed event time – threshold).' Events arriving after the watermark threshold are silently dropped.

Formula: Watermark = Max(Event Time seen so far) – Threshold Duration

10:00	10:05	10:10	10:15	10:20	10:25	10:30	10:35	10:40
✓ On time	✓ On time	✓ On time	✓ On time	✓ On time	⚠ Allowed	⚠ Allowed	✗ Dropped	✗ Dropped
Max event time observed = 10:35 Threshold = 10 minutes Watermark = 10:25 → Events with event time < 10:25 are dropped								

Figure: Watermark Timeline – events before the watermark threshold are dropped; state for closed windows is freed

Effect of Watermarks on State

Without watermark: State for all windows is held in memory forever. The state store grows unboundedly, eventually causing OutOfMemory errors.

With watermark: Once a window's end time falls below the watermark, its state is finalized and evicted from memory. State size is bounded.

Dropping Duplicates in Streams

Streaming sources like Apache Kafka provide at-least-once delivery semantics. This means the same event may be delivered more than once (due to retries, producer failures, or consumer restarts). Spark Structured Streaming provides `dropDuplicates()` to remove duplicate events based on one or more identifying columns.

How it works: Spark maintains a state store containing the set of previously seen record identifiers. When a new record arrives, its identifier is checked against the store. If already seen, it is dropped.

Deduplication with watermark: The watermark bounds the deduplication state — identifiers older than the watermark are evicted. This prevents the state from growing indefinitely but means very late duplicates (beyond watermark) may not be detected.

Without watermark: Deduplication state grows indefinitely as all previously seen identifiers must be retained.

Q6. Explain Spark Structured Streaming — its core concepts, the unbounded table model, and how it differs from Spark's legacy DStream API.

Answer:

What is Structured Streaming?

Structured Streaming is a scalable, fault-tolerant, and high-level stream processing engine built on top of the Spark SQL engine. Introduced in Spark 2.0, it represents a major evolution from the legacy DStream API (Spark Streaming). It allows developers to express streaming computations using the same DataFrame/SQL API used for batch processing.

The Unbounded Table Model

The fundamental mental model of Structured Streaming is that a live data stream is treated as a table that is continuously being appended with new rows. Each micro-batch appends new rows to this conceptual 'Input Table'. A query over this table produces a 'Result Table' which is updated on each trigger. The output policy (append, update, complete) controls what portion of the Result Table is written to the sink.

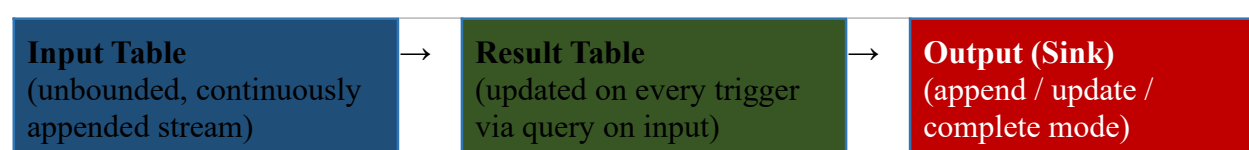


Figure: Structured Streaming Unbounded Table Model

Structured Streaming vs Legacy DStream API

Feature	DStream API (Legacy)	Structured Streaming
Introduced	Spark 1.x	Spark 2.0+
API	Low-level RDD-based DStream	High-level DataFrame/SQL API
Optimization	No automatic optimization	Catalyst optimizer + Tungsten
Event-Time Support	Limited / manual implementation	Native built-in support
Watermarking	Not supported	Fully supported
Fault Tolerance	WAL + checkpointing	Checkpointing + exactly-once semantics
Exactly-Once	Hard to guarantee	Built-in guarantee with right source/sink
Status	Maintenance mode (no new features)	Active development – recommended

Q7. What transformations are supported in Spark Structured Streaming? Clearly explain which operations are NOT supported and why. Describe the output modes.

Answer:

Transformations in Structured Streaming

Structured Streaming supports most of the same DataFrame transformations available in batch mode, because it processes each micro-batch as a small batch DataFrame internally. However, certain operations are fundamentally incompatible with the nature of unbounded, continuously arriving data.

Supported Transformations

Category	Supported Operations
Stateless	select, filter, where, withColumn, drop, map, flatMap, explode, union
Stateful (with watermark)	groupBy + agg, window aggregations, dropDuplicates, mapGroupsWithState
Stream-Static Joins	A stream joined with a static (batch) DataFrame or table
Stream-Stream Joins	Two streams joined with watermarks defined on both sides
SQL Queries	spark.sql() on a registered streaming view

Unsupported Operations and Reasons

Unsupported Operation	Reason for Restriction
Multiple streaming aggregations (chained groupBy)	Spark cannot maintain intermediate streaming state for chained aggregations. The output of first groupBy is not materializable for a second groupBy in streaming.
limit() and take()	Non-deterministic in a continuous stream — a fixed-count limit has no stable meaning when rows keep arriving.
distinct() without watermark	Requires tracking all previously seen rows forever, causing unbounded state growth.
sort() / orderBy() on stream	Global sorting requires seeing all data. Not possible on an unbounded, continuously arriving stream.
count() without groupBy	A single running count makes sense only in 'complete' output mode; unsupported in append mode.

Output Modes

The output mode defines what portion of the Result Table is written to the sink after each micro-batch trigger.

Output Mode	What is Written	Allowed For
append	Only NEW rows added to the Result Table since the last trigger. Rows are written ONCE and never updated.	Queries without aggregations; windowed aggregations with watermark
complete	The ENTIRE Result Table is rewritten to the sink after every trigger.	Aggregations only (e.g., running counts, totals)
update	Only rows that were UPDATED (changed) since the last trigger are written. More efficient than complete.	Aggregations with or without watermark

Q8. Explain all Input Sources supported in Spark Structured Streaming. Describe each source's characteristics, configuration options, and appropriate use cases.

Answer:

Overview

Spark Structured Streaming reads from input sources using the `spark.readStream` API. A source is a system that produces a continuous stream of data. Spark supports both production-grade sources and testing sources.

Source	Type	Key Characteristics	Use Case
Kafka	Production	Distributed, fault-tolerant, replayable. Supports topic subscription, consumer group offsets, earliest/latest offset configuration. Enables exactly-once with appropriate sink.	High-throughput event streams, IoT data, log aggregation, real-time analytics
File Source	Production	Monitors a directory for new files. Processes files atomically. Supports CSV, JSON, Parquet, ORC, and text. Schema must be defined explicitly.	Batch-arrival streaming files, log file ingestion, S3/HDFS file landing zone
Amazon Kinesis	Production	AWS-managed streaming service. Similar to Kafka. Requires Spark-Kinesis connector.	AWS cloud-native streaming pipelines
Socket Source	Testing only	Reads UTF-8 text from a TCP socket connection. Simple setup. No fault tolerance, no checkpointing support.	Local testing, educational demos, prototyping
Rate Source	Testing only	Generates rows at a configurable rate (rows per second). Each row has a timestamp and sequential value column.	Load testing, benchmarking, generating test data

Kafka Source – Key Configuration Options

Option	Description
<code>kafka.bootstrap.servers</code>	Comma-separated list of Kafka broker addresses (host:port)
<code>subscribe</code>	Comma-separated list of topics to subscribe to
<code>subscribePattern</code>	Regular expression pattern to match multiple topic names
<code>startingOffsets</code>	Where to start reading: 'earliest' (from beginning), 'latest' (new data only), or JSON offset map
<code>maxOffsetsPerTrigger</code>	Maximum number of offsets processed per micro-batch (rate limiting)
<code>failOnDataLoss</code>	Whether to fail when data is lost (e.g., topic deleted). Default true.

File Source – Key Configuration Options

Option	Description
path	Directory path to monitor for new files (HDFS, S3, local)
maxFilesPerTrigger	Maximum number of new files to process per micro-batch
latestFirst	Whether to process newest files first (default false)
cleanSource	Whether to delete or archive processed files ('delete', 'archive', 'off')

Q9. Explain all Output Sinks supported in Spark Structured Streaming. Describe fault tolerance guarantees, output modes, and configuration of each sink.

Answer:

Overview

A sink is the destination where Structured Streaming writes its results. Spark supports multiple built-in sinks and allows custom sinks via the `foreach` and `foreachBatch` APIs.

Sink	Fault Tolerance	Supported Output Modes	Best For
Console Sink	None (testing only)	append, complete, update	Debugging during development
File Sink	Exactly-once (atomic file commits)	append only	Writing results to HDFS, S3, or local storage as Parquet/CSV/JSON
Kafka Sink	At-least-once (idempotent writes possible)	append, complete, update	Publishing processed results back to Kafka topics
Memory Sink	None (in-memory table, lost on restart)	append, complete	Interactive querying during development (via SQL)
<code>foreach</code> Sink	Depends on implementation	append, complete, update	Custom row-by-row output to any system (e.g., a database, REST API)
<code>foreachBatch</code> Sink	Depends on implementation	append, complete, update	Custom batch-level output; enables multiple outputs; reuses batch DataFrame APIs

File Sink Details

Atomicity: Files are written to a temporary location and atomically moved to the final path only after the micro-batch is complete. This ensures no partial files are visible.

Exactly-once guarantee: The file sink achieves exactly-once semantics by tracking which files have been committed in the checkpoint directory.

Compaction: Over time, many small files accumulate. Periodic compaction jobs (batch Spark jobs) are recommended to merge small files for efficient reading.

foreachBatch Sink – Key Use Cases

- Writing to multiple sinks (e.g., both a database and a Kafka topic) from a single stream
- Applying batch DataFrame transformations not available in streaming (e.g., multiple aggregations)
- Implementing custom deduplication or upsert logic before writing to a database

Trigger Types for All Sinks

Trigger Type	Description	Latency
Default (unspecified)	Processes each micro-batch as soon as the previous one finishes	Seconds
Fixed Interval	Processes a micro-batch at a specified fixed interval (e.g., every 10 seconds)	Seconds
Once	Processes all available data in a single	Minutes (one-time)

	micro-batch, then stops the query	
AvailableNow	Similar to Once but processes data in multiple micro-batches until caught up, then stops	Minutes
Continuous	Experimental. Processes records as they arrive without batching. Asynchronous checkpointing.	Milliseconds

Q10. Explain the end-to-end architecture of a Spark Structured Streaming application. Describe the role of each component — source, query, state store, checkpoint, sink — and explain fault tolerance mechanisms.

Answer:

End-to-End Architecture Overview

A Spark Structured Streaming application follows a clearly defined pipeline from data ingestion to output. Understanding each component's role is essential for building reliable, high-performance streaming systems.

SOURCE <i>Kafka / File / Kinesis / Socket</i>	Produces continuous stream of data. Replayable sources (Kafka) are required for exactly-once guarantees.
▼	
OFFSET TRACKING <i>Source Offset Log (Checkpoint)</i>	Spark records the offsets of data processed in each micro-batch. On recovery, it re-reads from the last committed offset.
▼	
MICRO-BATCH ENGINE <i>DAG Scheduler + Task Scheduler</i>	Spark SQL engine processes each micro-batch as a small batch job. Transformations are optimized by the Catalyst Optimizer.
▼	
STATE STORE <i>In-Memory Key-Value Store (Checkpointed)</i>	Maintains aggregation state, deduplication records, and user-defined state across micro-batches. Backed by HDFS/S3.
▼	
CHECKPOINT DIRECTORY <i>HDFS / S3 / Local FS</i>	Stores offset logs, state store snapshots, and WAL (Write-Ahead Log). Enables recovery after failures.
▼	
SINK <i>File / Kafka / Database / Custom</i>	Receives the output of the query. Atomically commits results after each micro-batch. Idempotent sinks enable exactly-once delivery.

Figure: End-to-End Spark Structured Streaming Architecture

Fault Tolerance Mechanisms

Mechanism	How it Works	What it Protects Against
Offset Tracking	Source offsets are written to the checkpoint directory before processing begins (Write-Ahead Log style). On restart, Spark re-reads from the last committed offset.	Data loss when executor or driver crashes mid-batch
State Store Checkpointing	State store snapshots are written to HDFS/S3 after each micro-batch. On restart, the latest snapshot is restored.	State loss (aggregation results, deduplication records)

Idempotent Sinks	Sinks like the File Sink use atomic commits. Writing the same data twice produces the same result.	Duplicate writes when a batch is retried after failure
Replayable Sources	Sources like Kafka retain data for a configurable retention period. Spark can re-read from any past offset.	Re-reading data after a failure and recovery

Exactly-Once Guarantee

Spark Structured Streaming achieves end-to-end exactly-once processing when all three conditions are met: (1) the source is replayable (e.g., Kafka), (2) the state store is checkpointed, and (3) the sink is idempotent or supports transactional writes. Any single failure in this chain degrades the guarantee to at-least-once.